RELATÓRIO TÉCNICO DE AUDITORIA DE SEGURANÇA

Auditoria de Código-Fonte e Arquitetura — Planify AI

Avaliação sistemática de vulnerabilidades e vetores de risco nas 5 categorias críticas de aplicação web e APIs.

Projeto: Planify AI (SaaS Controle Financeiro)	**Data da Auditoria:** 30 de Outubro de 2026
Branch Auditada: develop (commit HEAD: 4c41fcf)	**Status Geral:** Ação Requerida (3 Críticas, 5 Altas)
Escopo: 100% dos Handlers de Rota, Server Actions e Componentes	**Metodologia:** White-Box Source Code Review

Nota Metodológica & Detecção de Stack

Antes do início dos testes, a stack do projeto foi catalogada e cada uma das cinco categorias de segurança foi mapeada para as características reais do código:

• **Linguagem & Framework:** TypeScript 5 sobre Next.js 16.2.9 (App Router) e React 19.

• **ORM & Banco de Dados:** Drizzle ORM sobre Neon Serverless PostgreSQL. Como não há RLS ativo no Postgres, o isolamento depende 100% da cláusula manual userId em cada query Drizzle.

• **Autenticação:** JWT customizado assinado via jose (HS256) em cookie HTTP-Only. Não há middleware global ativo (o arquivo middleware.ts foi deletado no histórico).

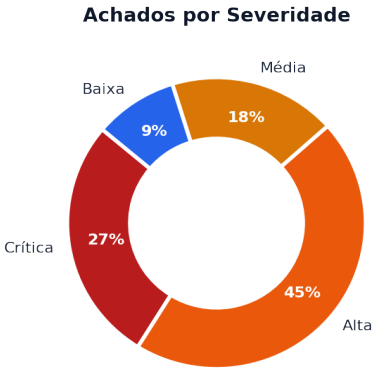
• **Modelo de Permissões:** Aplicação B2C monousuário; a tabela users não possui campos de papel (role ou isAdmin). O teste de controle de privilégio concentrou-se na autorização de rotas privilegiadas de sistema (cron jobs e webhooks).

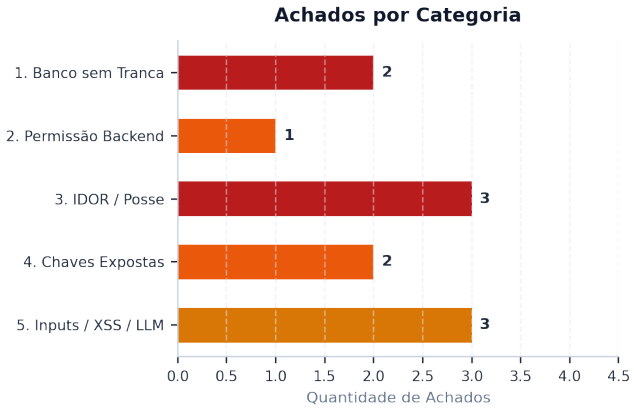
• **Integrações Críticas:** Webhook do Telegram (mensagens/voz/faturas em PDF) e APIs do Google Gemini.

1. Resumo Executivo

A auditoria de código-fonte cobriu a totalidade dos endpoints de API (src/app/api/...), Server Actions (src/app/**/actions.ts), páginas protegidas e rotinas utilitárias. Foram identificados **11 achados de segurança verificados no código real**, sem falsos positivos ou especulações. Destacam-se **3 vulnerabilidades críticas** que permitem personificação de usuários, manipulação de saldo e sequestro da integração do Telegram.

3 CRÍTICAS	5 ALTAS	2 MÉDIAS	1 BAIXA	6 CONTROLES
Acesso/mutação imediata	Falhas de autorização e injeção	Configuração e resiliência	Higiene de repositório	Módulos auditados íntegros





2. Pontos Fortes e Pontos Fracos Centrais

PONTOS FORTES (CONTROLES ROBUSTOS)

- **Validação de Posse no Core de Ações:** Os módulos accounts, categories e subscriptions aplicam rigorosamente and(eq(table.id, id), eq(table.userId, session.user.id)) em updates e deletes.
- **Proteção Server-Side em Todas as Páginas:** Todas as páginas privadas (Dashboard, Contas, Cartões, Transações, etc.) checam getSession() no servidor antes de renderizar.
- **Validação Estrita com Zod:** Todas as Server Actions validam inputs com schemas Zod tipados.
- **Segredos de Terceiros Protegidos:** O arquivo .env.local está no .gitignore e nenhuma chave de API foi commitada no histórico git.
- **Prevenção de XSS Tradicional:** Não há dangerouslySetInnerHTML nem eval no front.

PONTOS FRACOS (RISCOS CENTRAIS)

- **Fallback Hardcoded de Segredo JWT:** A aplicação adota chave fixa se a variável não estiver setada, permitindo forjar sessões de qualquer usuário.
- **Ausência de Autenticação no Webhook do Telegram:** Não há conferência do secret token, permitindo chamadas externas maliciosas.
- **IDOR e Manipulação de Saldo via Bot:** Callbacks de confirmação e cancelamento alteram registros e saldos bancários sem checar titularidade.
- **Vinculação de Conta por ID Estático:** O comando /start <userId> vincula a conta sem comprovação de posse ou senha.
- **Bypass em Rotas de Cron:** Se CRON_SECRET estiver ausente, a checagem é pulada.

3. Tabela de Achados Detalhados por Categoria

A tabela a seguir consolida todos os 11 achados de segurança identificados, indicando a severidade, a localização exata no código-fonte e o resumo do risco operacional associado.

Severidade	Arquivo e Linha	Categoria e Resumo do Achado
CRÍTICA	src/lib/auth.ts:19	**Chaves Expostas:** Fallback estático hardcoded 'super_secret_key_planify' permite forjar JWTs e obter login total sem senha.
CRÍTICA	src/app/api/telegram/route.ts:106, 167, 191, 245, 317	**IDOR / Autenticação:** Webhook do Telegram sem verificação de segredo e callbacks que deletam, confirmam e alteram saldo de outros usuários.
CRÍTICA	src/app/api/telegram/route.ts:624-635	**IDOR / Posse:** Vinculação de conta via /start <userId> aceita UUID estático sem OTP, permitindo Account Takeover no Telegram.
ALTA	src/app/api/cron/subscriptions/route.ts:21-27 src/app/api/cron/daily-report/route.ts:11-17	**Permissão Backend:** Lógica de checagem condicional (&&) pula autenticação se CRON_SECRET não for configurado.
ALTA	src/app/api/import-invoice/route.ts:90-101, 127	**IDOR / Banco sem Tranca:** Importação de fatura PDF aceita creditCardId de terceiro sem validar posse do usuário da sessão.
ALTA	src/app/cards/actions.ts:20-32, 78-89	**IDOR / Banco sem Tranca:** Cadastro e atualização de cartão aceita autoPayAccountId de outro usuário sem validação de posse.
ALTA	src/lib/telegram.ts:19, 52 src/lib/telegram-utils.ts:130, 165	**Inputs / XSS:** Injeção de Markdown em mensagens enviadas ao Telegram causa falha de envio (HTTP 400) e permite inserção de links forjados.
ALTA	src/app/api/ai-report/route.ts:40-54 src/app/api/ai-insights/route.ts:63-73	**Inputs / LLM:** Prompt Injection em relatórios de IA devido à interpolação direta de descrições e categorias sem delimitação contextual.
MÉDIA	next.config.ts:28	**Inputs / CSP:** Política de Content-Security-Policy permissiva com 'unsafe-inline' e 'unsafe-eval' anula defesa em profundidade.
MÉDIA	src/db/index.ts:5	**Chaves / Config:** Fallback silencioso de conexão do banco para URL fictícia mascara falhas de ambiente em vez de falhar no startup.
BAIXA	src/lib/test-local.ts:29, 51, 57	**Chaves / Repositório:** Arquivo rastreado no Git expõe caminhos locais do sistema operacional e ID real de usuário do Telegram.

4. Recomendações Priorizadas (Plano de Remediação)

PRIORIDADE 1 (P1) — CRÍTICA / IMEDIATA (24-48 horas)

1. **Eliminar Fallback de JWT_SECRET:** Forçar erro de inicialização imediato (throw new Error(...)) caso JWT_SECRET não esteja presente no ambiente. Rotacionar a chave em produção imediatamente.
2. **Blindar Webhook do Telegram:** Configurar e exigir o cabeçalho X-Telegram-Bot-Api-Secret-Token em todas as requisições para /api/telegram.
3. **Eliminar IDOR nos Callbacks do Telegram:** Associar cada transação/parcelamento buscado em callbacks ao usuário cadastrado com o chatId do remetente.
4. **Vincular Telegram com Código Seguro:** Substituir /start <userId> por um token temporário de 6 dígitos gerado na sessão do dashboard web com validade de 10 minutos.

PRIORIDADE 2 (P2) — ALTA (Próxima Sprint / 1 semana)

5. **Corrigir Guardas de Cron:** Rejeitar a execução se authHeader !== `Bearer ${process.env.CRON_SECRET}` ou se CRON_SECRET for vazio, independentemente do ambiente.
6. **Validar Posse de Cartões e Contas:** Nas rotas de importação de fatura e Server Actions de cartões, validar que creditCardId e autoPayAccountId pertencem ao session.user.id.
7. **Higienizar Markdown do Telegram:** Aplicar função de escape para caracteres reservados do Telegram antes de interpolar textos em mensagens.
8. **Delimitar Contexto de IA:** Utilizar blocos JSON e delimitadores estruturados ao enviar entradas de usuários para a API Gemini.

PRIORIDADE 3 (P3) — MÉDIA E BAIXA (Ciclo Contínuo)

9. **Endurecer CSP:** Remover 'unsafe-eval' e avaliar uso de nonces para scripts inline.
10. **Excluir src/lib/test-local.ts:** Remover arquivo com dados pessoais e caminhos locais do histórico git.

5. Issues Prontas para o GitHub

Abaixo estão os textos completos em Markdown de cada issue acionável, prontos para copiar e colar no GitHub Issues. Os achados menores de mesma natureza foram agrupados em issues coesas para evitar ruído.

````
--- ISSUE 1 ---
Título: [Segurança] Tornar JWT_SECRET obrigatório e eliminar fallback estático inseguro

Labels sugeridas: security, critical, backend, auth

### Descrição do Problema
O arquivo `src/lib/auth.ts` define a chave secreta de assinatura JWT com um fallback estático
`process.env.JWT_SECRET || 'super_secret_key_planify'`. Da mesma forma, `src/db/index.ts` utiliza fallback de
string fictícia.

### Por que é explorável
Se `JWT_SECRET` não estiver presente ou falhar ao carregar no container ou Vercel, o backend inicializa
normalmente usando o segredo público `'super_secret_key_planify'`. Qualquer invasor pode assinar tokens JWT
forjados contendo o ID de qualquer usuário e autenticar-se como administrador/vítima sem fornecer credenciais.

### Evidência de Código
`src/lib/auth.ts:19`:
```ts
const secretKey = new TextEncoder().encode(process.env.JWT_SECRET || 'super_secret_key_planify');
```

### Impacto
Comprometimento total da autenticação e quebra do isolamento de todos os inquilinos (Account Takeover
generalizado).

### Sugestão de Correção
1. Lançar exceção imediata no startup se `JWT_SECRET` não estiver presente:
```ts
if (!process.env.JWT_SECRET || process.env.JWT_SECRET.length < 32) {
throw new Error('FATAL: JWT_SECRET must be configured with at least 32 characters.');
}
const secretKey = new TextEncoder().encode(process.env.JWT_SECRET);
```
2. Rotacionar imediatamente o segredo de produção.

### Critérios de Aceite
- [ ] O backend rejeita a inicialização com erro explícito caso `JWT_SECRET` esteja ausente ou vazio.
- [ ] O fallback hardcoded foi completamente removido do código.
- [ ] Teste unitário valida a rejeição de inicialização sem a variável configurada.
--- FIM ISSUE 1 ---
````

````
--- ISSUE 2 ---
Título: [Segurança] Validar Secret Token no Webhook do Telegram e corrigir IDOR em callbacks

Labels sugeridas: security, critical, api, telegram, idor

### Descrição do Problema
O endpoint `POST /api/telegram` não valida o cabeçalho `X-Telegram-Bot-Api-Secret-Token`. Além disso, os
manipuladores de `callback_query` (`confirm_inst_`, `cancel_inst_`, `toggle_type_`, `confirm_`, `cancel_`)
consultam objetos no banco puramente pelo ID, sem checar se pertencem ao remetente da mensagem.

### Por que é explorável
Um atacante pode forjar requisições HTTP para `/api/telegram` enviando payloads de botões com IDs de outros
usuários. Isso permite cancelar transações alheias, alterar tipos de lançamentos e até debitar/estornar saldos
de contas bancárias de terceiros (`accounts.balance`).

### Evidência de Código
`src/app/api/telegram/route.ts:106, 191, 245, 317`:
```ts
const txRes = await db.select().from(transactions).where(eq(transactions.id, txId));
```

### Impacto
Destruição e adulteração de dados financeiros de outros usuários e fraude em saldos bancários.

### Sugestão de Correção
1. Validar no topo da rota:
```ts
const secretToken = req.headers.get('x-telegram-bot-api-secret-token');
if (secretToken !== process.env.TELEGRAM_WEBHOOK_SECRET) {
return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
}
```
````

````
2. Nos callbacks, obter o usuário vinculado ao `chatId` e filtrar:
```ts
const txRes = await db.select().from(transactions)
.where(and(eq(transactions.id, txId), eq(transactions.userId, user.id)));
```

### Critérios de Aceite
- [ ] Requisições sem o header secreto correto recebem 401 Unauthorized.
- [ ] Callbacks com IDs de transações pertencentes a outro usuário são rejeitados com erro e não alteram o
banco.
- [ ] Testes automatizados cobrem a verificação de posse em todas as ações de callback.
--- FIM ISSUE 2 ---
````

````
--- ISSUE 3 ---
Título: [Segurança] Corrigir vinculação insegura de contas Telegram via /start

Labels sugeridas: security, critical, auth, telegram

### Descrição do Problema
O comando `/start ` aceita o UUID do usuário em texto claro para associar a conta do Planify AI ao
`telegramChatId` do remetente sem exigir código temporário, confirmação de sessão ou senha.

### Por que é explorável
O link exibido na tela web (`src/app/page.tsx:324`) expõe o UUID. Qualquer usuário malicioso que conheça ou
descubra esse UUID pode enviar `/start ` ao bot a partir de seu próprio Telegram e assumir a conta, recebendo
relatórios com o saldo financeiro diário e operando transações em nome da vítima.

### Evidência de Código
`src/app/api/telegram/route.ts:629`:
```ts
await db.update(users).set({ telegramChatId: chatId }).where(eq(users.id, userId));
```

### Impacto
Sequestro do canal de comunicação Telegram, vazamento de saldos e transações financeiras.

### Sugestão de Correção
1. Implementar um fluxo de OTP (One-Time Password) ou token assinado com validade curta (ex: 10 minutos):
- O usuário clica em "Conectar Telegram" no dashboard;
- O backend gera um token assinado (HMAC) ou código de 6 dígitos temporário salvo no banco/Redis: `/start
link_ABC123`;
- O bot valida o código único e expira o token imediatamente após o uso.

### Critérios de Aceite
- [ ] O comando `/start` não aceita mais UUIDs brutos de usuários.
- [ ] A vinculação só ocorre mediante token assinado/OTP temporário com tempo de expiração curto (<= 15 min).
- [ ] Tentativas de reutilizar o token geram erro.
--- FIM ISSUE 3 ---
````

````
--- ISSUE 4 ---
Título: [Segurança] Corrigir guarda lógica de autorização em rotas de Cron Jobs

Labels sugeridas: security, high, backend, api

### Descrição do Problema
As rotas `GET /api/cron/subscriptions` e `GET /api/cron/daily-report` utilizam uma condicional que só rejeita a
chamada caso `process.env.CRON_SECRET` exista. Se a variável não estiver definida ou se `NODE_ENV` não for
`'production'`, a rota roda sem exigir credenciais.

### Por que é explorável
Se um operador esquecer de configurar `CRON_SECRET` na Vercel ou se a rota for chamada em ambiente de
staging/preview, qualquer pessoa pode chamar `GET /api/cron/subscriptions` e acionar débitos em massa e avanço
de cobranças em todas as assinaturas do banco.

### Evidência de Código
`src/app/api/cron/subscriptions/route.ts:21-27`:
```ts
if (
process.env.NODE_ENV === 'production' &&
process.env.CRON_SECRET &&
authHeader !== `Bearer ${process.env.CRON_SECRET}`
) {
return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
}
```

### Impacto
Execução não autorizada de débitos em massa e envio descontrolado de mensagens via Telegram.
````

```
### Sugestão de Correção
Exigir sempre a presença do segredo (fail-closed):
```ts
const expectedSecret = process.env.CRON_SECRET;
if (!expectedSecret || authHeader !== `Bearer ${expectedSecret}`) {
return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
}
```

Critérios de Aceite
- [ ] Se `CRON_SECRET` não estiver setado, a rota retorna 401 Unauthorized.
- [ ] Requisições sem o header `Bearer ` recebem 401 Unauthorized em todos os ambientes.
--- FIM ISSUE 4 ---
```

```
--- ISSUE 5 ---
Título: [Segurança] Validar titularidade de creditCardId e autoPayAccountId

Labels sugeridas: security, high, idor, backend

### Descrição do Problema
Na rota `POST /api/import-invoice` e nas Server Actions de cartões (`addCreditCard`, `updateCreditCard`), os parâmetros `creditCardId` e `autoPayAccountId` são aceitos sem validação de que pertencem ao usuário autenticado (`session.user.id`).

### Por que é explorável
Um usuário pode importar despesas vinculando-as a cartões de crédito de outros usuários, corrompendo relatórios e faturas alheias. Também pode definir uma conta de débito automático pertencente a terceiro.

### Evidência de Código
`src/app/api/import-invoice/route.ts:90`:
```ts
const cardRes = await db.select().from(creditCards).where(eq(creditCards.id, creditCardId));
```

Impacto
Injeção de transações no contexto de terceiros e violação de isolamento entre usuários.

Sugestão de Correção
Adicionar a checagem explícita de `userId`:
```ts
const cardRes = await db.select().from(creditCards)
.where(and(eq(creditCards.id, creditCardId), eq(creditCards.userId, session.user.id)));
if (cardRes.length === 0) {
return NextResponse.json({ error: 'Cartão de crédito não encontrado ou não autorizado' }, { status: 403 });
}
```

Critérios de Aceite
- [ ] A importação de faturas falha com 403 caso o `creditCardId` pertença a outro usuário.
- [ ] `autoPayAccountId` só pode ser associado se a conta pertencer ao usuário autenticado.
--- FIM ISSUE 5 ---
```

```
--- ISSUE 6 ---
Título: [Segurança] Sanitizar caracteres especiais em mensagens com parse_mode Markdown no Telegram

Labels sugeridas: security, high, telegram, xss

### Descrição do Problema
O módulo de envio para o Telegram (`src/lib/telegram.ts`) utiliza `parse_mode: "Markdown"` sem escapar caracteres especiais (`*`, `_`, `[`, `]`, `\``, `(`) contidos nos dados fornecidos pelo usuário (`description`, `category`).

### Por que é explorável
Descrições com caracteres Markdown não balanceados provocam erro HTTP 400 da API do Telegram (`can't parse entities`), quebrando o fluxo da transação. Além disso, permite injeção de links forjados em mensagens enviadas pelo bot.

### Evidência de Código
`src/lib/telegram-utils.ts:165`:
```ts
return `${icone} *Revisão de Transação*\n\nTipo: *${tipo}*\nDescrição: *${data.description}*...`;
```

Impacto
Negação de serviço funcional no registro de transações via Telegram e risco de phishing por injeção de links.

Sugestão de Correção
```

````
Criar função de escape para o modo Markdown ou migrar para `parse_mode: "HTML"` utilizando entidades seguras
(`&`, `<`, `>`):
```ts
function escapeMarkdown(text: string): string {
return text.replace(/([_*[\]()~`>#+\-=|{}.!])/g, '\\$1');
}
```

### Critérios de Aceite
- [ ] Transações com nomes contendo `_`, `*`, `[` ou `]` são enviadas sem erro pela API do Telegram.
- [ ] Links Markdown maliciosos inseridos em descrições não são renderizados como hiperlinks clicáveis.
--- FIM ISSUE 6 ---
````

```
--- ISSUE 7 ---
Título: [Segurança] Endurecer defesas contra Prompt Injection e reforçar Content Security Policy

Labels sugeridas: security, high, ai, csp, frontend

### Descrição do Problema
1. O texto das transações e faturas é concatenado diretamente nas mensagens de prompt do Google Gemini em
`src/app/api/ai-report/route.ts` e `src/lib/gemini.ts`.
2. A política de CSP em `next.config.ts:28` permite `'unsafe-inline'` e `'unsafe-eval'`.
3. O arquivo `src/lib/test-local.ts` rastreado no git expõe caminhos absolutos locais do desenvolvedor e o ID
real de chat do Telegram.

### Impacto
Manipulação de saídas da IA via dados controlados pelo usuário e redução das defesas do navegador contra XSS.

### Sugestão de Correção
1. Utilizar delimitadores XML/JSON claros no prompt do Gemini com instruções explícitas de que os dados não
contêm comandos do sistema.
2. Remover `'unsafe-eval'` do CSP em `next.config.ts`.
3. Remover `src/lib/test-local.ts` do repositório ou adicioná-lo ao `.gitignore`.

### Critérios de Aceite
- [ ] Prompts do Gemini utilizam tags delimitadoras de contexto estruturado.
- [ ] A diretiva `'unsafe-eval'` é removida da configuração do CSP.
- [ ] O arquivo `src/lib/test-local.ts` é expurgado do versionamento git.
--- FIM ISSUE 7 ---
```